

Computación Gráfica 2024

Unidad 1:

Introducción a la Computación Gráfica

UNIVERSIDAD NACIONAL DEL LITORAL
FACULTAD DE INGENIERÍA Y CIENCIAS HÍDRICAS

FICH

UNL

Disclaimer: Estas notas se comparten en calidad de borrador y sin haber sido revisadas adecuadamente. Seguramente contengan errores de tpeo y de hortografía.

Objetivos de esta introducción:

- Hablar de qué es la "Computación Gráfica" en general, qué parte vamos a estudiar en esta materia y hasta dónde vamos a llegar.
- Dar una idea muy básica de cómo funciona un sistema gráfico y las dos principales familias de algoritmos de renderizado. Armar un mapa/esquema general del funcionamiento.

Que luego cuando profundicemos las distintas etapas de este proceso en cada unidad podamos ubicar cada tema dentro de este mapa general.

- En el proceso de introducir esto vamos a dejar también algunas definiciones/conceptos básicos importantes.

Al final de la presentación se aclara qué es lo más importante de saber/estudiar de todo lo mencionado, porque muchas cosas en esta unidad se muestran solo como ejemplos, para dar contexto, o para interesar al alumno, pero no es necesario recordarlas.

¿QUÉ ES "COMPUTACIÓN GRÁFICA"?

- Conjunto de técnicas para "comunicar" información de forma gráfica
 - Usualmente 3D
- ¿Qué implica?
 - Organizar la información
 - **Generar una imagen**
 - Animar/manipular/interactuar

La computación gráfica es el conjunto de técnicas utilizadas para "comunicar" información de forma gráfica. Puede ser 2D o 3D. Usualmente al decir "computación gráfica" se piensa primero en cosas 3D (ejemplos: juegos, CADs, películas animadas), y de hecho son las más interesantes. Pero muchas de ellas tienen su base en versiones 2D, y muchas aplicaciones no tan obvias de estas técnicas son 2D (por ejemplo: una ventana de una aplicación de gestión, o una página web son en general contenido 2D que hay que poner en pantalla y eventualmente manipular).

Modelar/organizar la información: ¿Cómo representa matemáticamente ese mundo virtual que voy a querer simular/dibujar? ¿Cómo el programa guarda esta información? ¿Qué estructuras de datos u organización de la memoria resulta más eficiente? ¿Qué aproximaciones se utilizan? ¿Cómo es la interfaz para que el usuario defina esta información?

Generar una imagen: Esta es la parte central y la que más estudiaremos en la materia. Algoritmos para tomar ese modelo del ítem anterior y generar una imagen que aproxime lo mejor posible cómo se vería eso en realidad.

Animar/manipular/interactuar: La mayoría de las aplicaciones que nos interesa tiene que ver con software interactivo: el usuario interactúa con la imagen generada para modificar el modelo o la escena y que la imagen se actualice para reflejarlo.

¿QUÉ NO ES?

• NO es **Usar** herramientas para hacer:

- animaciones
- juegos
- simulaciones
- CADs
- etc.

! *Sí es **hacer** esas herramientas*

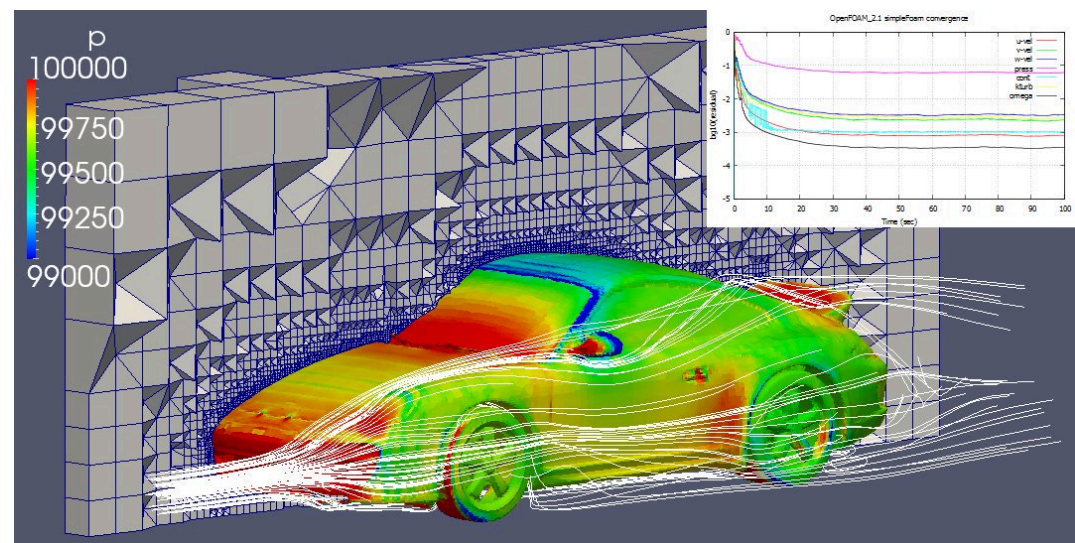
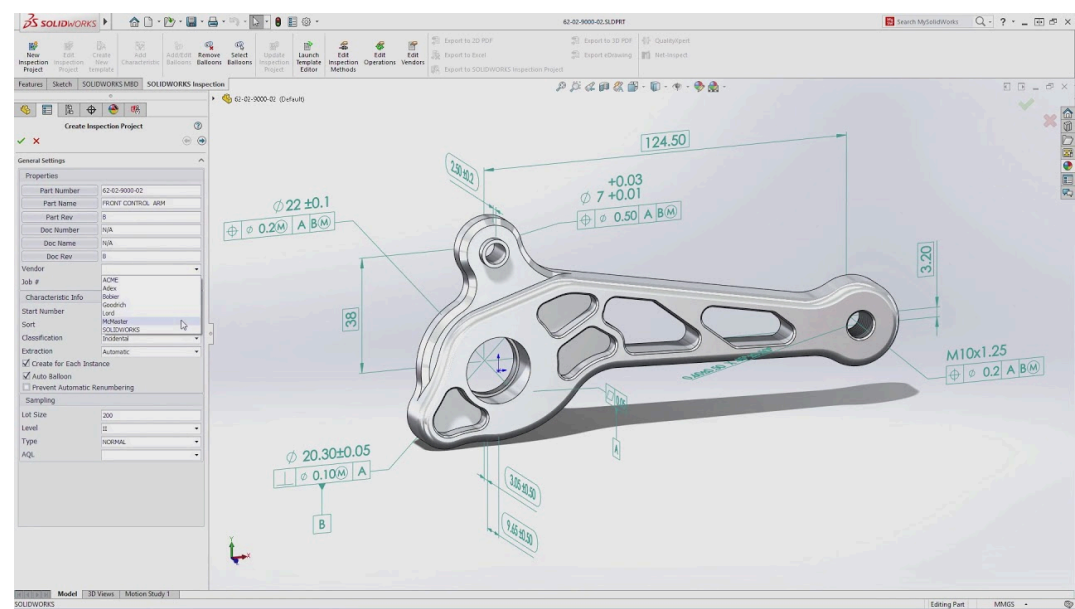
• NO es Análisis/procesamiento de imágenes

No es por ejemplo usar autocad (eso lo hace un arquitecto o un diseñador industrial por ej), blender/maya/3d studio (eso lo hace un diseñador gráfico).

Sí es "saber hacer" esas herramientas. Vamos a ver las técnicas que hay detrás, cómo funcionan o cómo se programan (partes de) esas herramientas.

No es análisis/procesamiento de imágenes: Eso va en otra materia. Cosas como: ¿cómo identificar dónde hay una cara en una foto? ¿cómo aplicarle filtros a una fotografía? o ¿cómo reducir el ruido? etc.

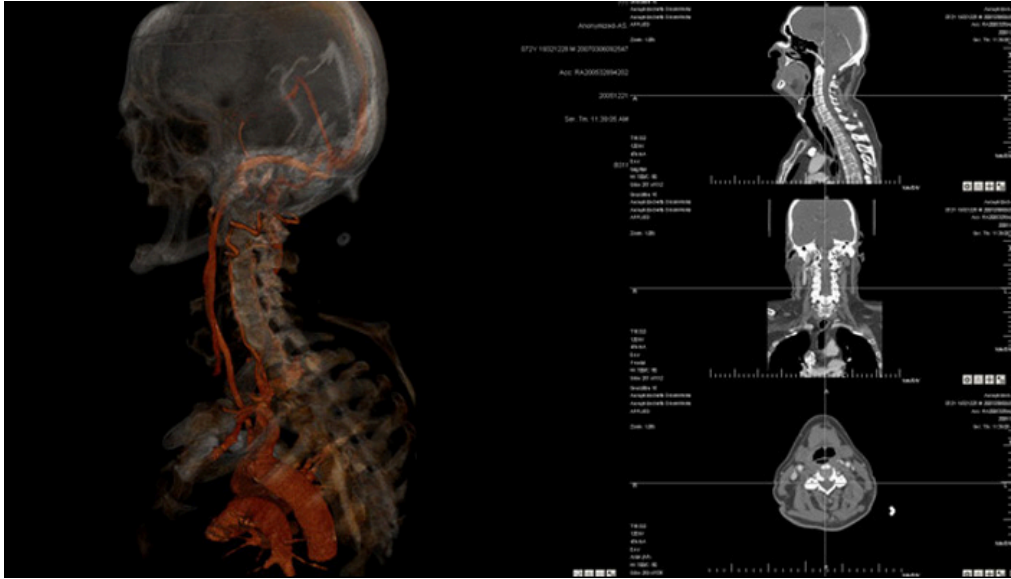
EJEMPLOS



Videojuegos, cine, herramientas CADs y de diseño, simulación computacional, visualización científica, visualización médica arte, arquitectura, diseño gráfico, realidad virtual, realidad aumentada...

Hasta el "simple" texto de cualquier ventana, página, o aplicación utiliza muchas técnicas de computación gráfica tanto para modelar la forma de los caracteres, como para "dibujarlos" correctamente en pantalla.

EJEMPLOS



¿HASTA DÓNDE VAMOS A LLEGAR EN LA MATERIA?



Vamos a hacer cosas que parecen ridículamente simples al lado de los ejemplos anteriores

Fundamentos y Obsolescencia:

En cada ejemplo hay cientos de técnicas combinadas, y solo vamos a tener tiempo de analizar unas pocas de las más simples.

Este campo avanza muy rápido, no tendría tanto sentido estudiar muchas técnicas avanzadas muy específicas en detalle porque probablemente ya no se usen cuando se reciban.

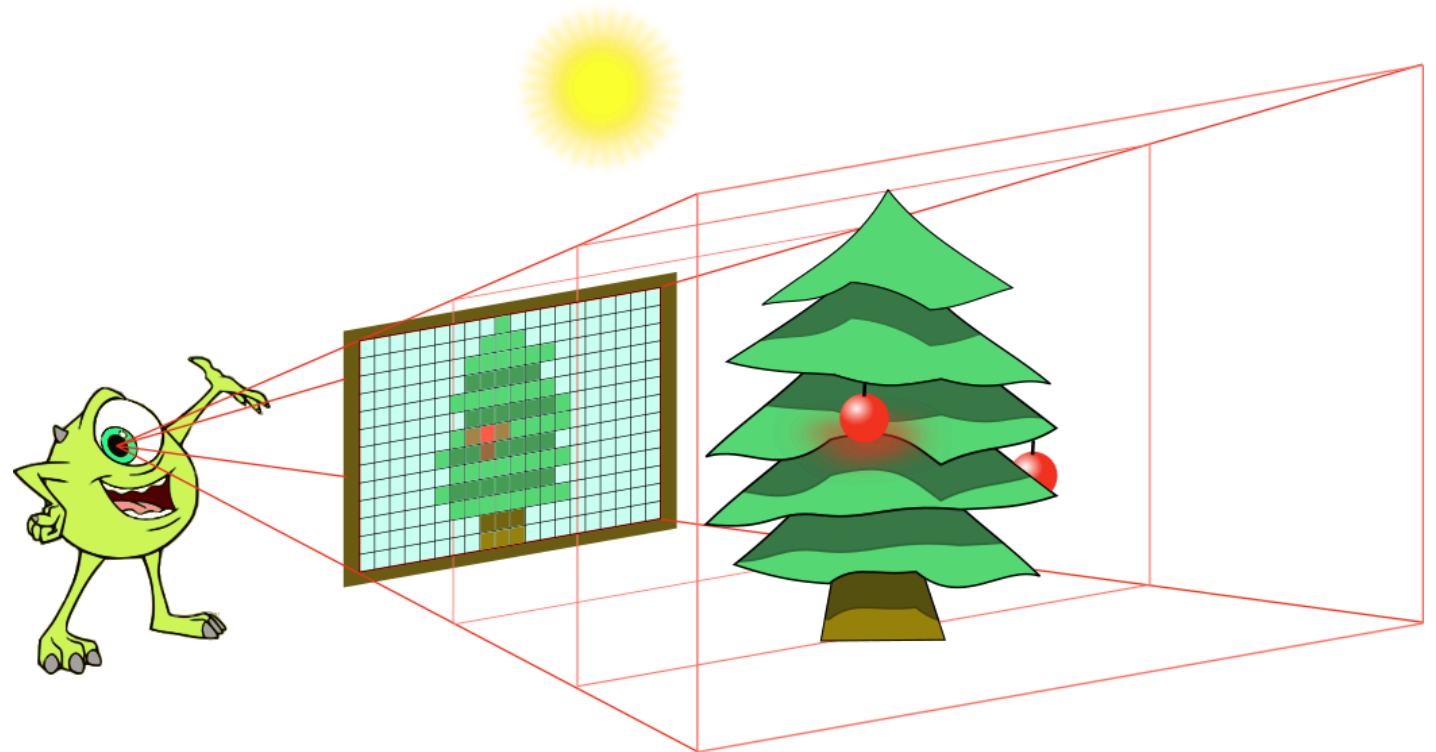
Nos vamos a centrar en las bases, los fundamentos, que son más duraderos, y son los que permiten luego entender el resto cuando les toque.

RENDERING

- Transformación de una **escena** en una imagen

- Escena:

- Objetos (forma, color, textura, etc).
- **Iluminación** (luces y ambiente)
- Cámara/Ojo



"Representar" sería la traducción al castellano de "rendering", pero es más común usar "rendering" o "renderizado", aún en material en español.

Es el proceso de tomar la descripción de la "Escena" y generar una imagen. La escena incluye generalmente: los objetos, las luces y la cámara.

Objetos: Lo principal es describir la forma, ubicación, tamaño, etc de los objetos. Esto es importante para determinar en qué lugar de la imagen va cada uno. Vamos a hablar más abajo de cómo aproximamos la forma real. Además de la forma, el color/material/textura contribuyen para determinar cómo se ve. No es lo mismo una esfera de plástico rojo, de metal pitado de rojo, de plastilina roja, etc.

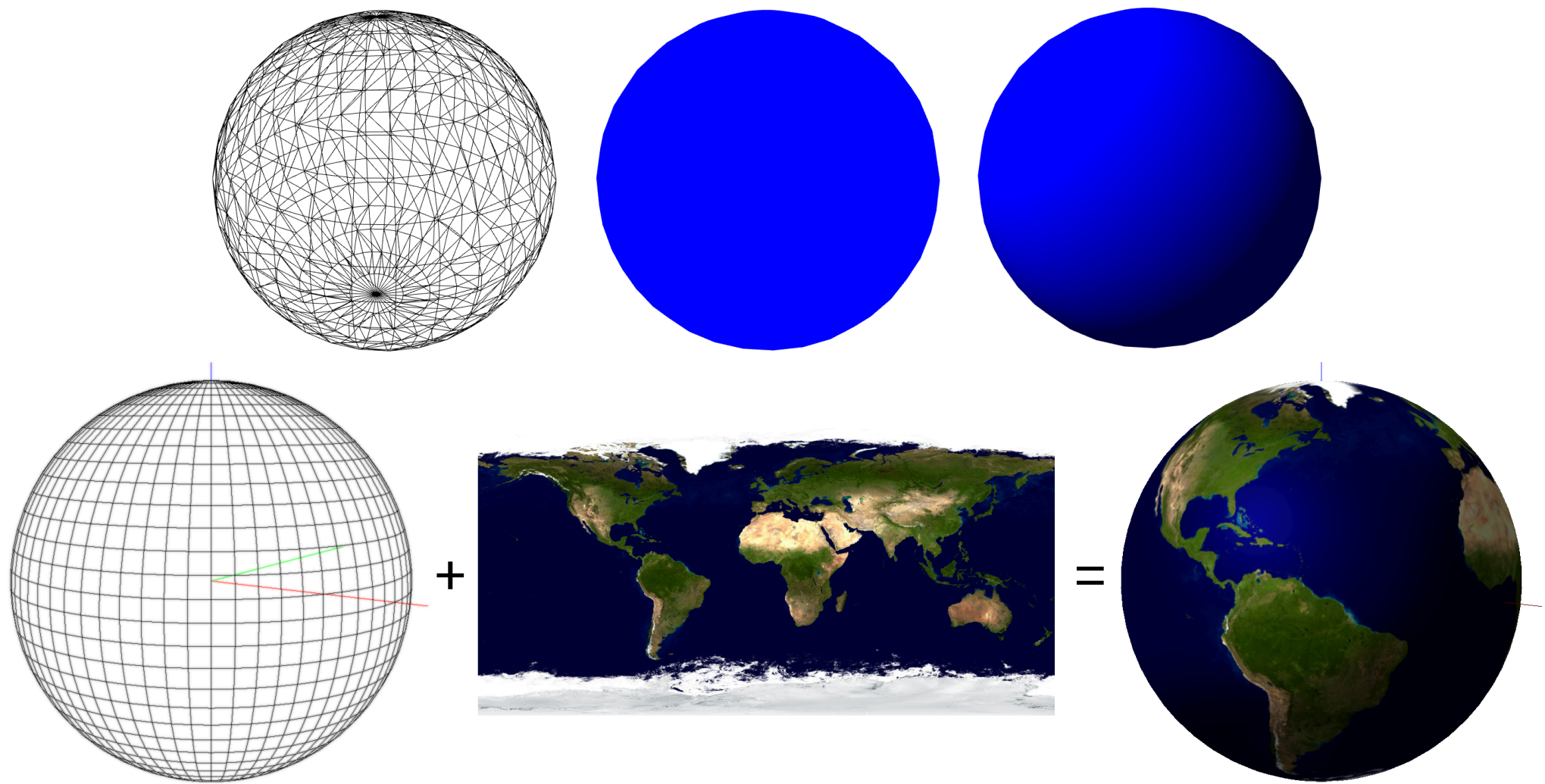
Luces: Los objetos se ven si les llega la luz. Una esfera roja a la que no le llega la luz se vería negra o casi negra. Una esfera de metal puede mostrar en algún punto un brillo blanco sin importar su color de base. Es fundamental describir las fuentes de luz de la escena.

El ambiente también puede alterar la visualización (ej, si hay niebla o humo)

Cámara/Ojo: Hay que saber la posición y orientación de la cámara, pero también otros parámetros que definen la perspectiva (la apertura, el zoom, etc) u otras propiedades de la misma.

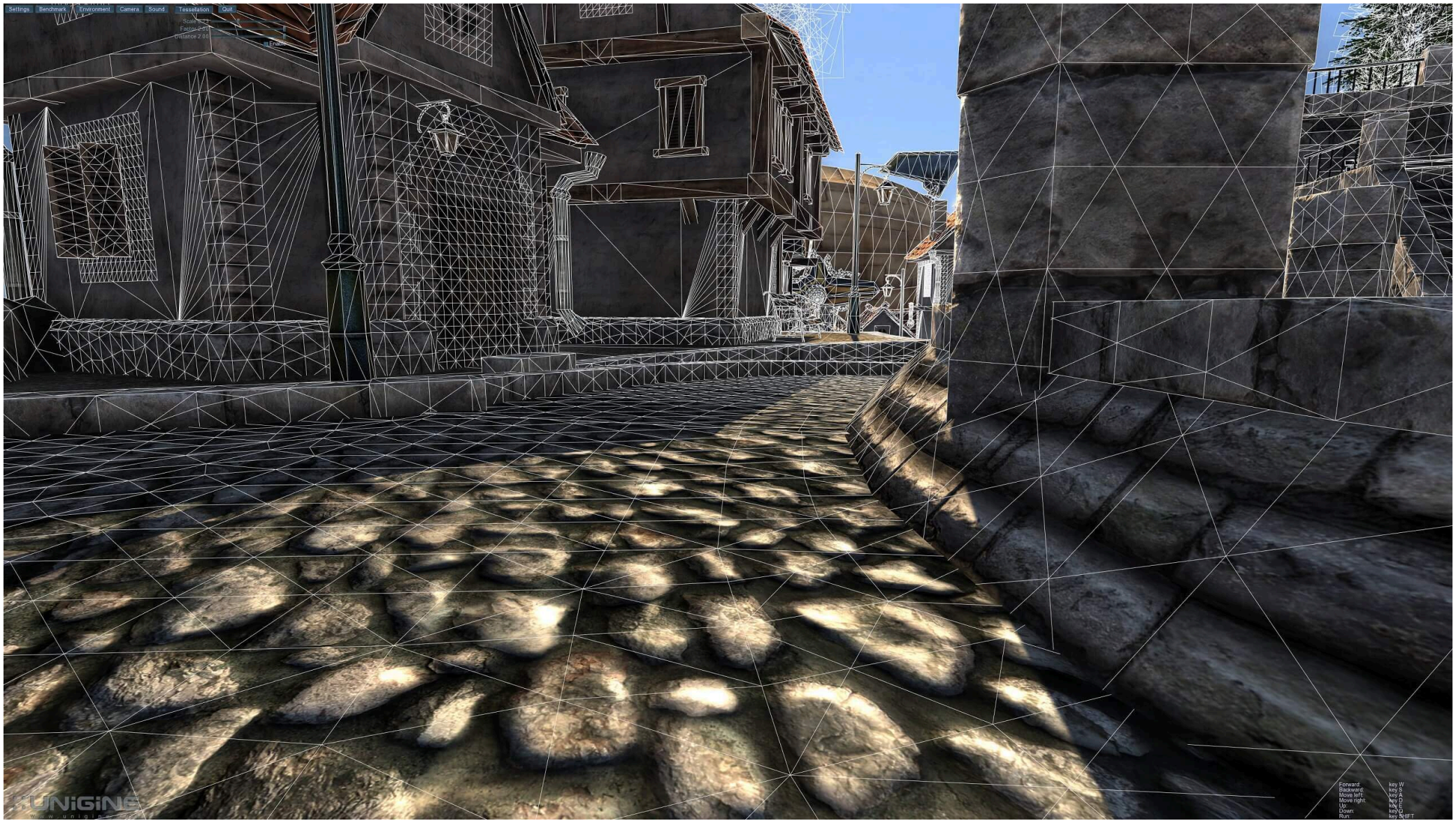
Cuanto más realista o detallado deba ser el resultado, más compleja/completa será la información de la escena. La descripción de cada una de estas 3 cosas entonces puede variar mucho en nivel de detalle, y requerir otra información adicional además de la mencionada.

OBJETOS



Primitivas habituales: **triángulos** y cuadriláteros

MODELADO: B-REP (EJEMPLO GRUESO)



MODELADO: B-REP (EJEMPLO FINO)



Usualmente modelamos solo la "superficie" de los objetos.

No vemos el interior. Aún si los partimos, vemos una nueva superficie interna.

Lo habitual es "aproximar" la superficie mediante "primitivas".

Una primitiva es una forma geométrica muy simple que aproxima una partecita de la superficie.

En el 99% de los casos, la primitiva es el triángulo.

También se utilizan muchas veces cuadriláteros para definir la forma, pero en realidad casi siempre terminan partiéndose en dos triángulos cada uno para procesarlos.

Cada superficie se aproxima mediante cientos o miles de triángulos "pegados" y pintados de forma que se disimulen las uniones.

Una escena de un videojuego actual puede tener millones de triángulos.

RENDERING: FOTOREALISTA VS NPR

► Fotorealista



► No-Fotorealista



Realista vs NPR

NPR = Non-photorealistic rendering = renderizado no fotorealista.

A veces el objetivo es una imagen lo más realista posible; mientras que otras veces se persigue una estética particular con fines artísticos (como en el ejemplo de la imagen) o para acentuar ciertas propiedades (como puede ser en visualización científica o médica, o en un cad que busca destacar la estructura).

RENDERING: EN TIEMPO REAL VS. OFFLINE

- ▶ Tiempo real
 - ▶ Juegos
 - ▶ CADs
 - ▶ Realidad aumentada
 - ▶ Realidad virtual
- ▶ Offline
 - ▶ Cine
 - ▶ Arquitectura
 - ▶ Visualización Científica

"En Tiempo Real" se dice cuando se tiene que generar la imagen "rápido" para dar la sensación de animación, o para que la aplicación responda rápidamente a los comandos del usuario.

¿Qué es rápido? depende del tipo de aplicación. Un juego necesita 30 o 60 cuadros por segundo para que se sienta "fluido". Un CAD o una visualización científica/médica puede tomar un poco más de tiempo y tal vez renderizar solamente alrededor de 10 cuadros por segundo sin perder la sensación de interactividad.

El renderizado es "offline" (o también "por lotes/batch") puede tomar mucho tiempo. Por ejemplo, cada fotograma de una película animada puede tardar horas o hasta días. Pero no es problema, una vez generado ya no habrá necesidad de modificarlo, la película luego se reproduce muchísimas veces igual.

Como las restricciones de tiempo son muy diferentes entre ambos tipos, los algoritmos que se utilizan habitualmente para cada uno también lo son. Veremos detalles más abajo, pero la materia se centrará más en algoritmos para tiempo real.

RASTER VS VECTOR

► Raster



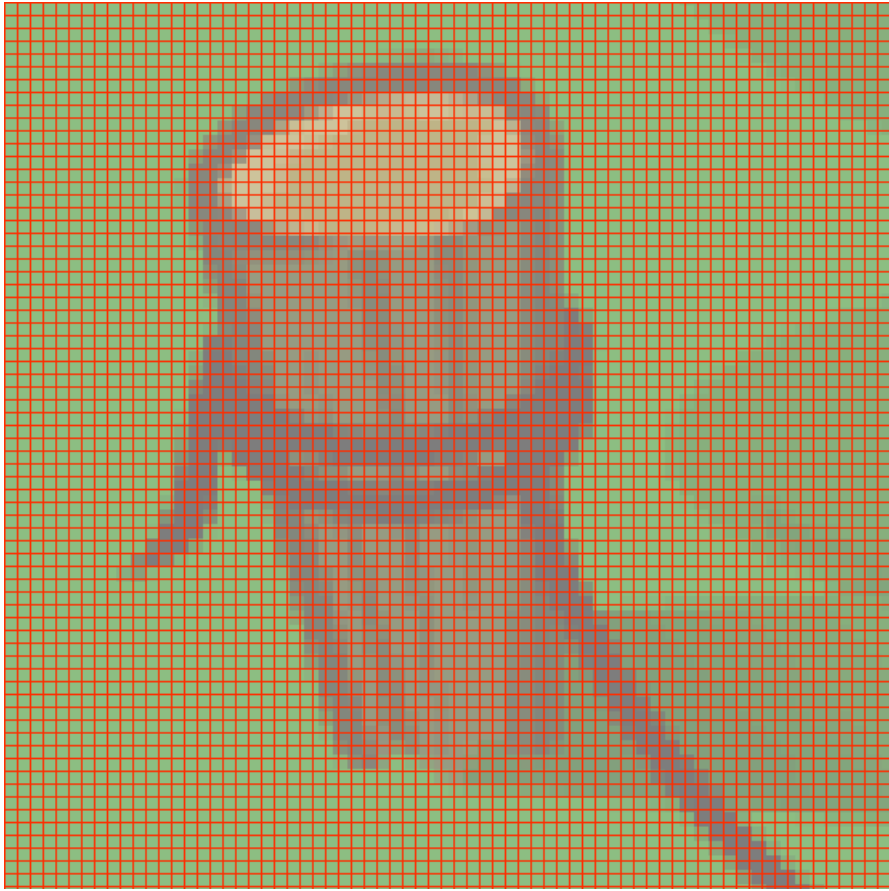
► Vector



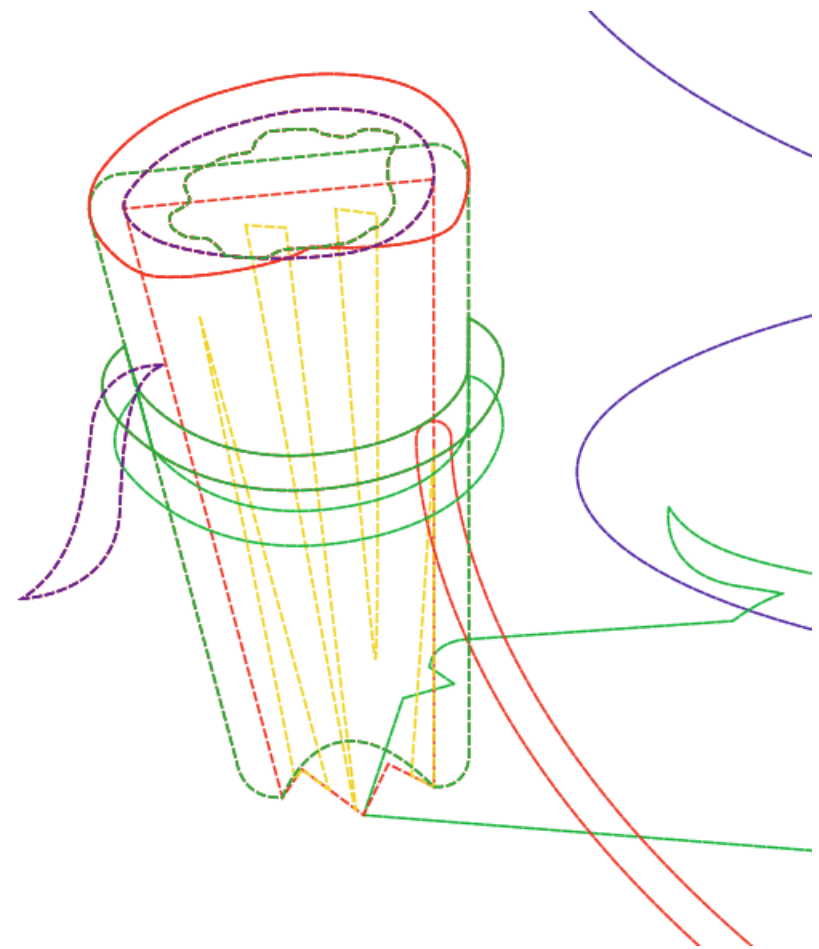
Ej: GIMP/Paint/Photoshop vs. Inkscape/Corel Draw/Illustrator

RASTER VS VECTOR

► Raster



► Vector



Esto ya no son dos tipos de rendering, esta clasificación aplica a cómo se representa o almacena información gráfica.

Una imagen vectorial está definida por ecuaciones/fórmulas matemáticas que describen la forma de las cosas (y propiedades de cada una, como el color).

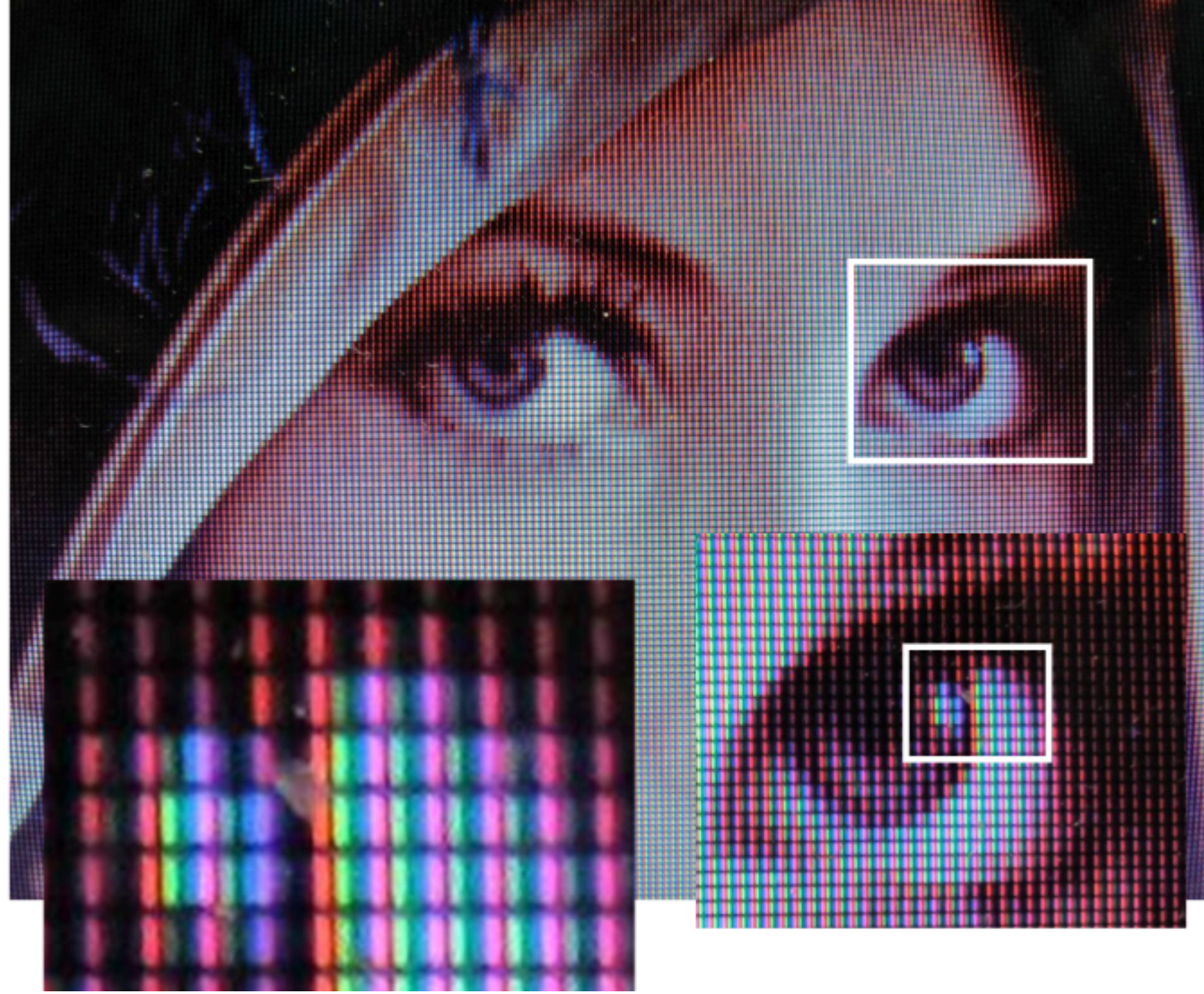
Estas fórmulas son continuas, por eso no importa el zoom que le hagamos nunca se verá "pixelada/serruchada". Software que trabaja vectorialmente: inkscape, corel draw, ilustrator, powerpoint, etc

Una imagen raster es una gran grilla/matriz donde cada casillero toma un color. A cada casillero de la grilla/elemento de la matriz se lo denomina "pixel". Cuanto más filas/columnas (más resolución/más chicos los píxeles) menos se nota que es una grilla y mejor se ve (más se disimula el serruchado).

Una foto digital es raster. Software que trabaja en modo raster: paint, gimp, photoshop, corel.

La mayoría de los programas que nos interesan guardan sus modelos vectorialmente. Pero todas las pantallas son raster (y la mayoría de las impresoras). La información vectorial debe convertirse a raster para ponerse en pantalla (estudiaremos esa conversión).

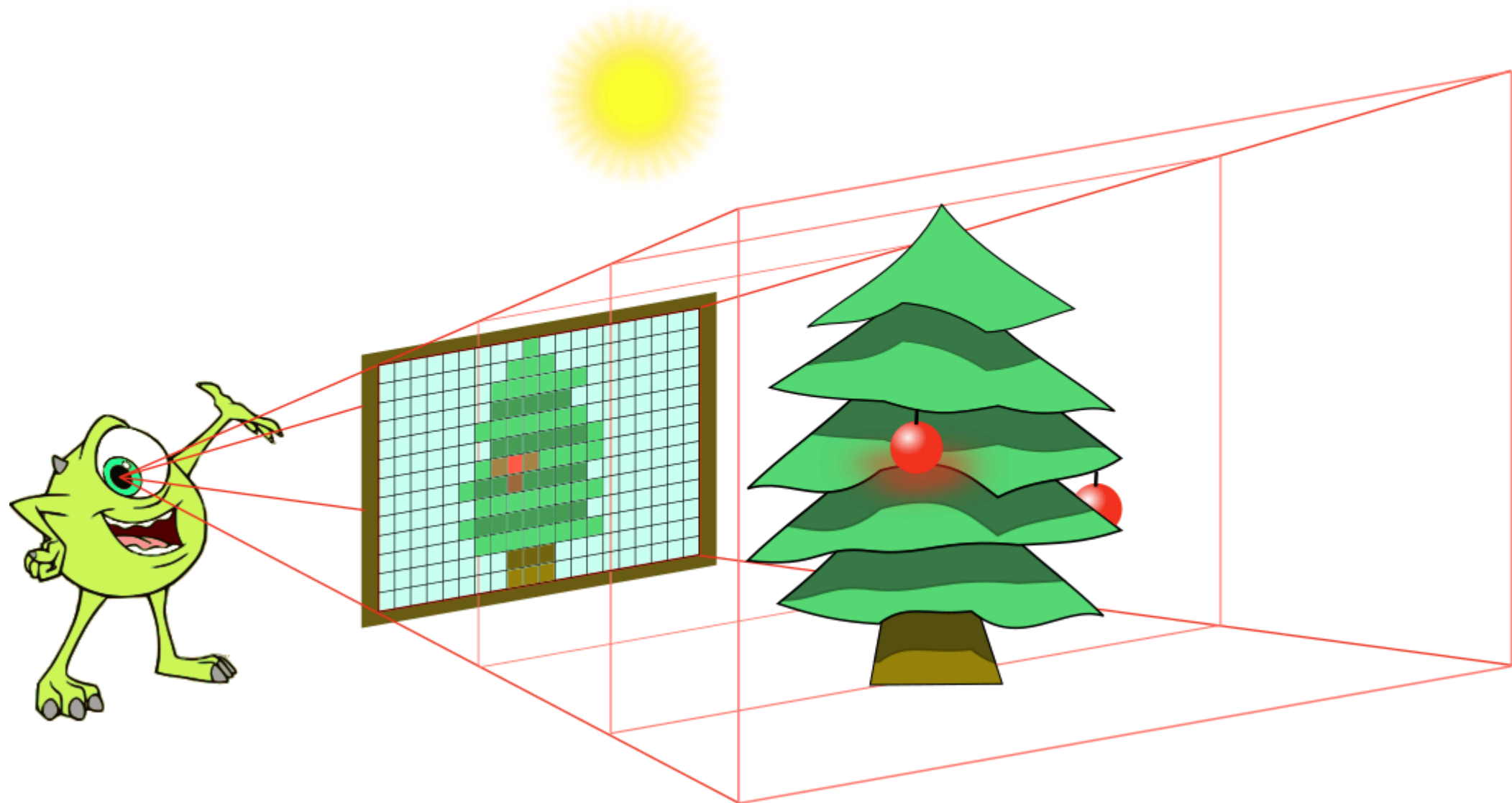
REPRESENTACIÓN DEL COLOR



En ambos casos (raster o vector), la forma de representar un color es mediante 3 valores que indican cuánto de rojo, cuánto de verde, cuánto de azul (¿por qué esos 3? lo veremos más adelante).

Ya no se pueden ver casi a simple vista en un monitor los 3 "subpíxeles". Sí con una cámara con macro, o en una pantalla gigante como las de publicidad en las esquinas.

RENDERING



Lo primero es determinar "qué" se ve y qué no se ve; y "dónde" se ve lo que se ve

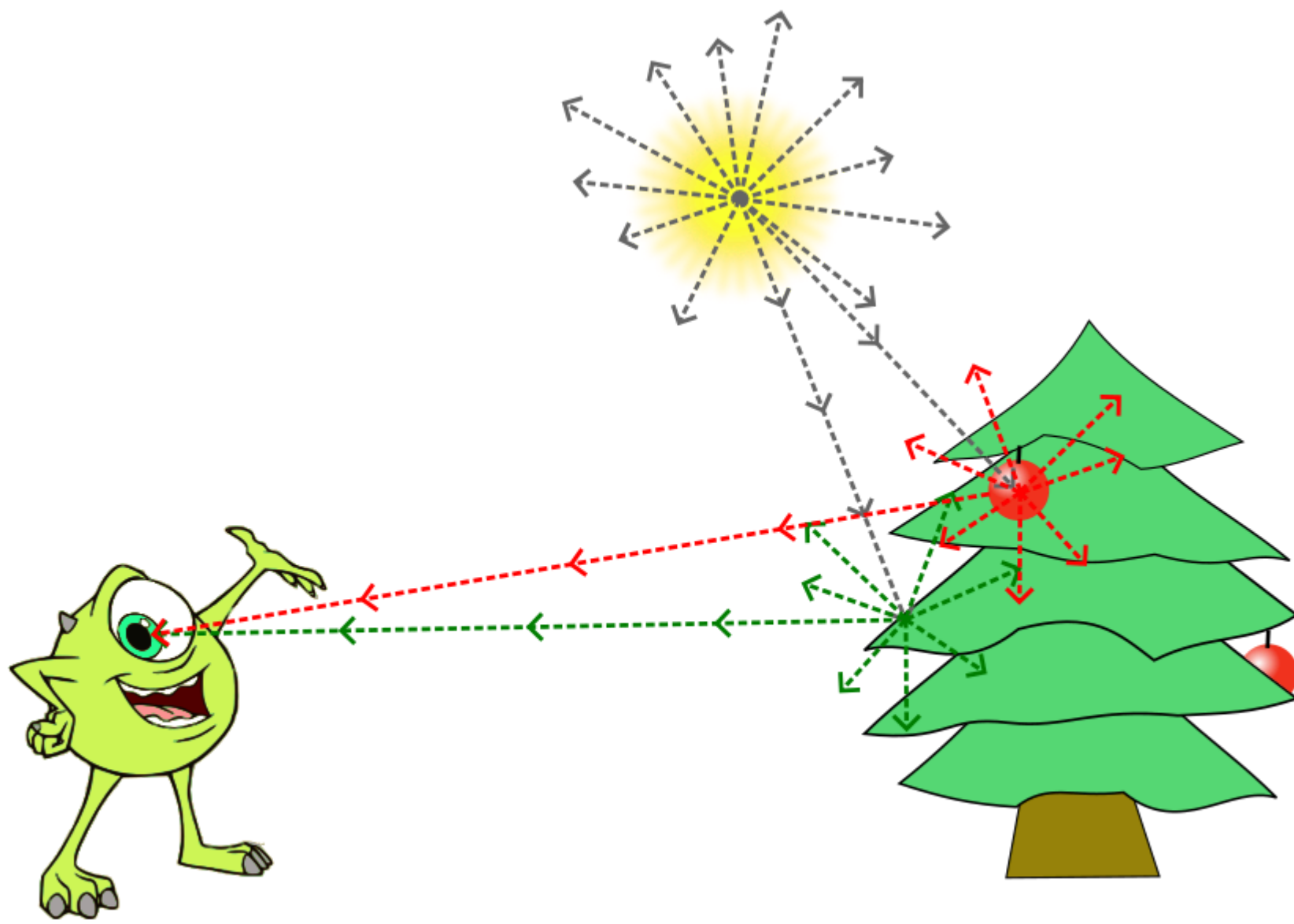
Esto da lugar a dos grandes familias de algoritmos:

Basados en "Raytracing": Por cada pixel a pintar, determinar qué veo tirando un rayo y calculando intersecciones contra los objetos.

Basados en "Rasterización": Por cada objeto a dibujar, determinar en qué lugar de la imagen se "proyectaría", y resolver de alguna forma qué hacer cuando la proyección de dos objetos cubre un mismo píxel.

En ambos casos, una vez determinada la relación objeto/píxel, hay que definir el color (que no necesariamente será el definido para el objeto, depende de cómo "le de" o no la luz y de su material).

¿CÓMO "FUNCIONA" LA REALIDAD?



Parece lo más natural pensando en cómo "funciona" la naturaleza/la realidad.

Vemos las cosas, porque las fuentes de luz emiten luz, y la luz va rebotando hasta que eventualmente parte de esa luz llega a nuestros ojos. Nuestros ojos son sensores de luz.

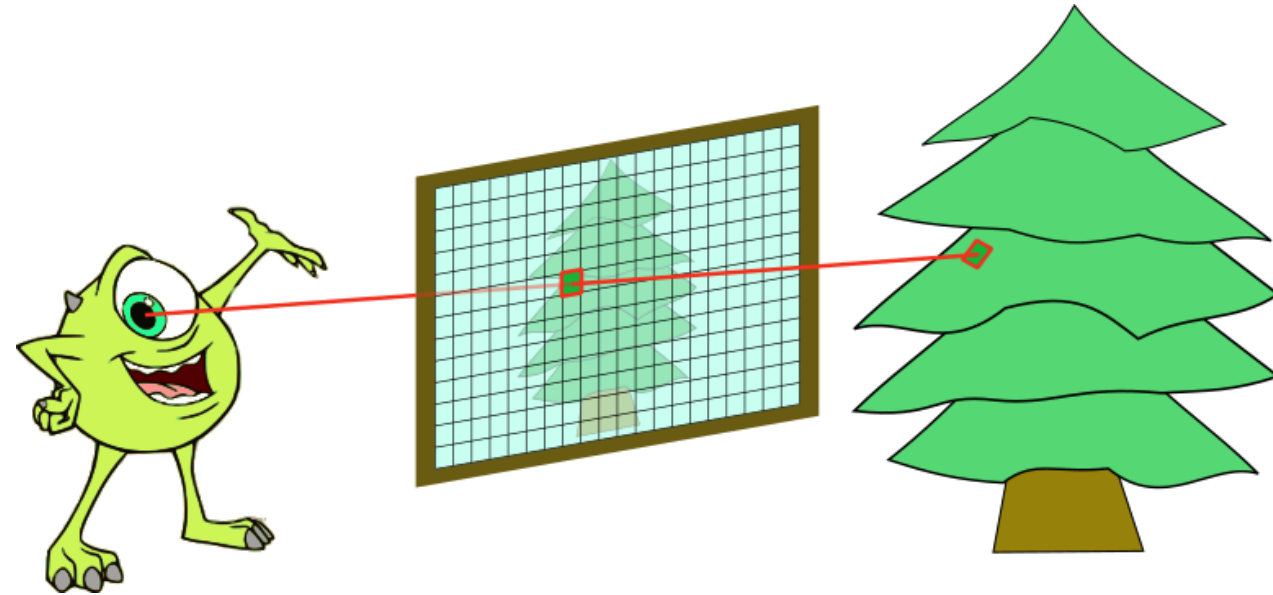
En cada rebote la luz cambia sus propiedades. Por ej: una lámpara led puede emitir luz blanca, que al rebotar sobre una superficie verde, "se convierte" en luz verde que llega al ojo.

Por ej: si veo un pedacito de una hoja verde de un árbol, es porque el sol emite luz blanca en muchas direcciones: parte de esa luz en la dirección hacia la hoja. La luz que llega a ese punto, choca y parte de ella rebota en muchas direcciones. La "parte" que rebota para todos lados es la "parte" verde (el resto es absorbido por la hoja). Entre las muchas direcciones en que se "desparrama" esa luz que venía del sol, estará la del ojo. Parte de esa luz que salió del sol entonces llega al ojo luego de uno o varios rebotes. El ojo tiene sensores que detectan esa luz e informan al cerebro: "acá hay algo verde".

Nota: casi siempre que en la realidad digo "muchas" en verdad es "infinitas" ya que estamos hablando de cosas continuas (por ej: las direcciones).

RENDERING: RAYTRACING

- **para cada pixel** (px) de la ventana
 - **para cada primitiva** (pr)
 - intersectar el rayo por px con pr
 - quedarse con la más cercana
 - calcular "color" y pintar

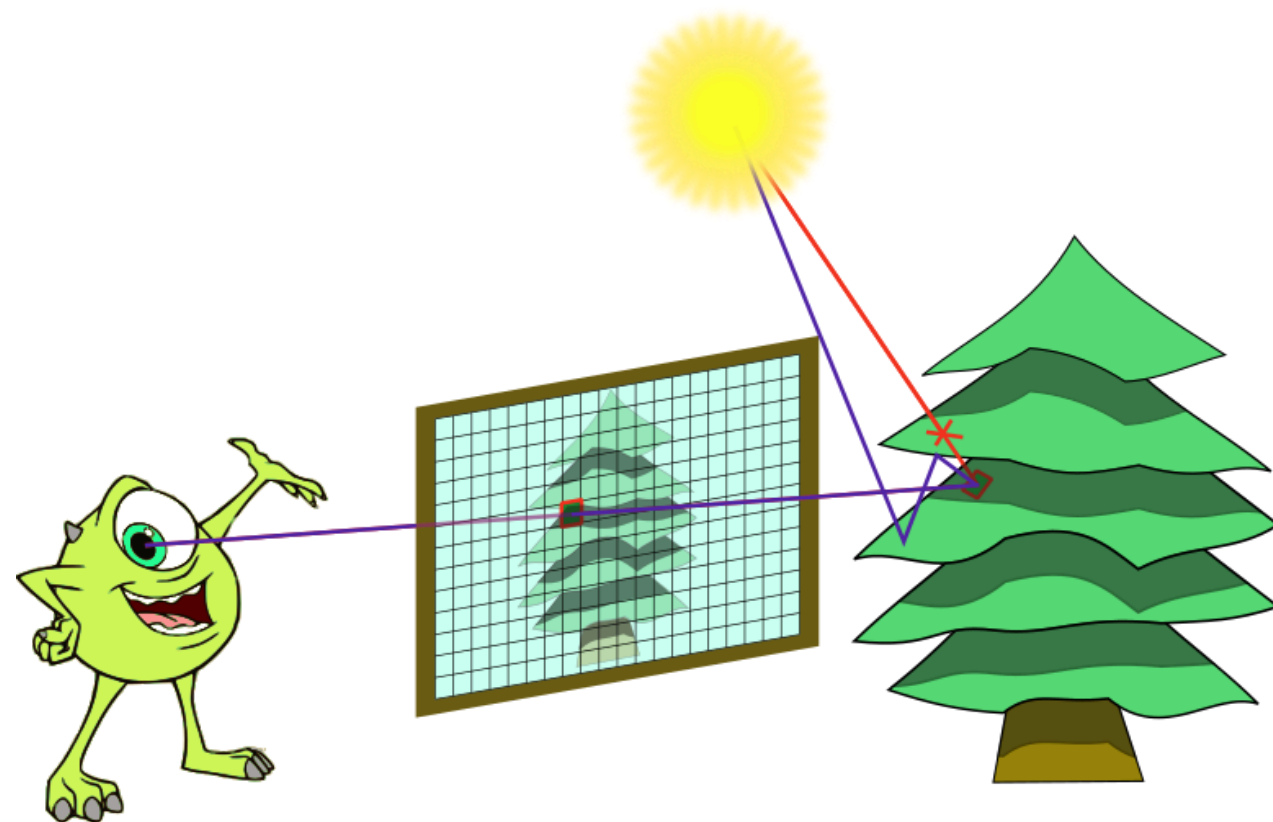


La mayoría de los rayos de luz (o mejor aún, de los fotones) no va a llegar al ojo: va a rebotar a cualquier otro lado o va a ser absorbido. Por esto, en el algoritmo hacemos el camino inverso: partimos con los rayos desde el ojo.

El principal cálculo que requiere entonces el algoritmo es la intersección del rayo contra los objetos. Por cada rayo hay que ver a qué "le pega", y de todos los que le pegue, quedarse con el primero (el más cercano a la cámara).

RENDERING: RAYTRACING

- Se puede reutilizar la "maquinaria" para iluminación y otros efectos.



Luego de determinado qué objeto se ve y dónde en la imagen, hay que determinar de qué color se ve (diremos "calcular la iluminación").

Si bien este segundo problema existe igual en rasterización, teniendo en marcha una maquinaria eficiente para tirar rayos y calcular intersecciones, se puede reutilizar para resolver esto:

Por ej: para saber si a un objeto le da la luz directamente (y entonces vemos su color real, o hasta un brillo si es metálico), o no (entonces se verá mucho más oscuro), podemos "simplemente" tirar un rayo desde el objeto hacia la luz y ver si le pega a algo en medio (algo que le haga sombra).

Podemos capturar muchísimos efectos de la realidad: sombras, sombreado, reflexiones, refracciones, caústicas, iluminación indirecta, anti-aliasing, motion-blur, profundidad de campo, etc.

La mayoría de estas cosas no se obtienen fácilmente con la alternativa basada en rasterización.

RENDERING: RAYTRACING

- Algoritmo (en principio) muy simple.
- Resultados muy **realistas**.
- Proceso muy **costoso**:
 - Miles de rayos contra miles de primitivas
- Utilizado históricamente para **renderizado offline**.

Dado que en una escena típica hoy en día hay cientos de miles, o hasta millones de triángulos, encontrar a cuál le pega primero por fuerza bruta es costosísimo.

Además, para que el resultado sea bueno se necesitan muchísimos rayos por pixel para empezar, y generar recursivamente muchos más para la iluminación.

El algoritmo es en esencia muy simple, sacando el "pequeño" (enooooorme) detalle de la eficiencia. Con poco (intersección rayo/primitiva) podemos prácticamente todo.

Por el alto realismo/detalle es el algoritmo más usado para renderizado off-line (ej: cine, arquitectura)

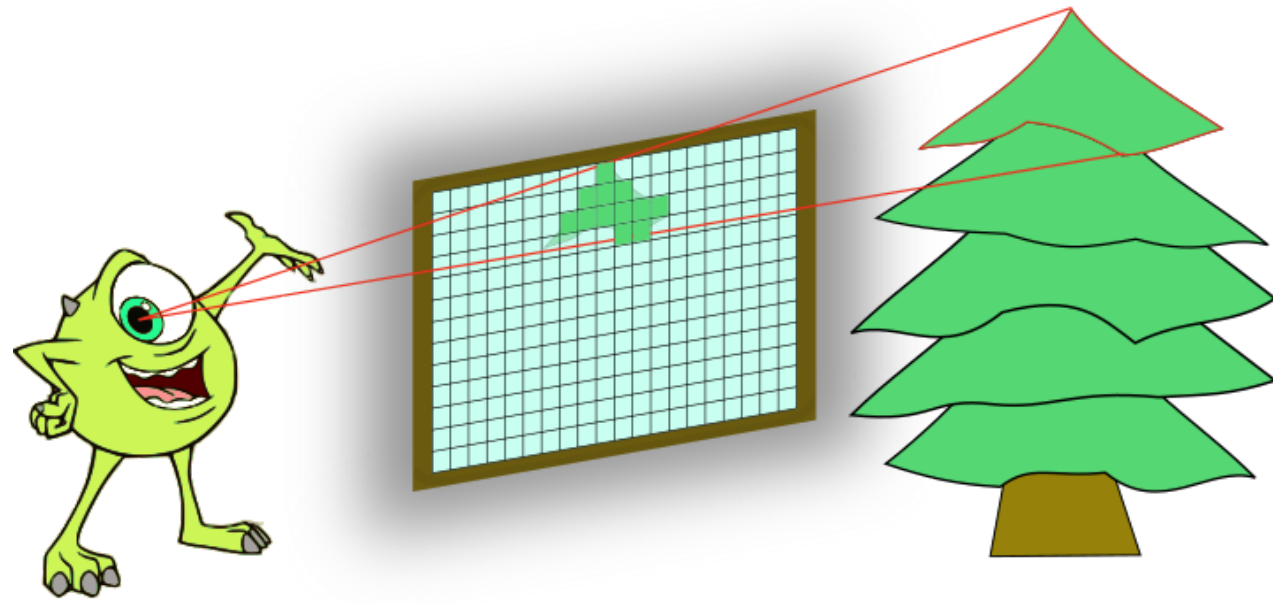
Por el alto costo/tiempo, históricamente no se ha usado para tiempo real (no era posible)

Hace muy poco (circa 2018/2019), entre la mejora de las técnicas, el avance del hardware y la capacidad de cómputo, y la proliferación de la inteligencia artificial para disimular lo que faltaba, empezó a ser posible pensar en raytracing en tiempo real.

Hoy en día muchos juegos combinan rasterización y raytracing, no hay todavía aplicaciones donde sea 100% raytracing (más allá de alguna prueba de concepto), pero estamos bastante más cerca.

RENDERING: RASTERIZACIÓN

- **para cada primitiva** (pr) de la escena
 - **para cada fragmento (px) que cubre**
 - determinar si es visible
 - calcular "color" y pintar



Invertimos los loops:

Raytracing era: por cada pixel, tiro un rayo y veo a qué primitiva le pega. El loop externo es el "por cada pixel", en "veo a qué le pega" está el loop interno (busca entre todas las primitivas).

En Rasterización: por cada primitiva, la proyecto en la imagen y veo qué píxeles cubre. El loop externo es el "por cada primitiva", y como la primitiva en pantalla puede ocupar muchos píxeles, el "veo qué píxeles cubre" es el loop interno que los recorre.

RENDERING: RASTERIZACIÓN

- Proceso (básico) **poco costoso**.
 - cada triángulo se procesa "solo".
- Resultado pobre
 - No captura muchos efectos (ej:sombras,reflejos).
 - Estas cosas se "truclean" con algoritmos ad-hoc.
- El paso "determinar si es visible" no es trivial
- Es el más utilizado para **tiempo real**

La ventaja y el problema es que cada primitiva se procesa independientemente de las demás.

Ventaja porque lo hace muuuy barato, por eso se prefiere para tiempo real (o en cualquier caso donde no importe tanto el realismo).

Desventaja porque mirandolos de uno, individualmente, no se puede capturar muchos efectos. Ej: para saber si a un triángulo hay que pintarlo iluminado o en sombra necesitaría saber si hay otro que le tape la luz.

Mirandolos de a uno, ni siquiera es trivial encontrar la forma de que si dos triángulos se proyectan sobre el mismo pixel, prevalezca el que corresponda. El "determinar si es visible" no es trivial, lo veremos en la unidad de buffers.

Por todo esto el resultado inicial es mucho menos realista, y el método requiere de muchos trucos adicionales para "truchar" todos los efectos que se pierde. Cada "efecto" o detalle es un truco diferente (un dolor de cabeza diferente).

La materia gira mayormente alrededor de la técnica de "rasterización", ya que el objetivo es poder realizar aplicaciones interactivas (tiempo real).

EL PIPELINE DE RASTERIZACIÓN

- Versión básica (solo render, etapas generales):



- Lo ejecuta la GPU

? ¿quién alimenta al pipeline?

Un "pipeline" es una secuencia de pasos encadenados, donde la salida de uno es la entrada del siguiente, cada paso va transformando la información.

Para rasterizar, podemos simplificarlo en 3 grandes pasos:

1. Por cada primitiva (en general una terna de puntos definiendo un triangulito), ver dónde se proyecta en el plano de la imagen (esto está en el mundo vectorial);
2. Una vez que el triángulo está ubicado en relación a la imagen, determinar qué píxeles cubre (esta es efectivamente la "rasterización", aquí pasamos de lo "vectorial" (el triangulito) a lo raster (conjunto de "fragmentos")).

El "fragmento" está muy emparentado con un píxel. Representa una posición a pintar en esa matriz/imagen raster. Pero que todavía no se pinta y que tiene algo de información adicional. Por ahora diremos que es un "candidato a píxel".

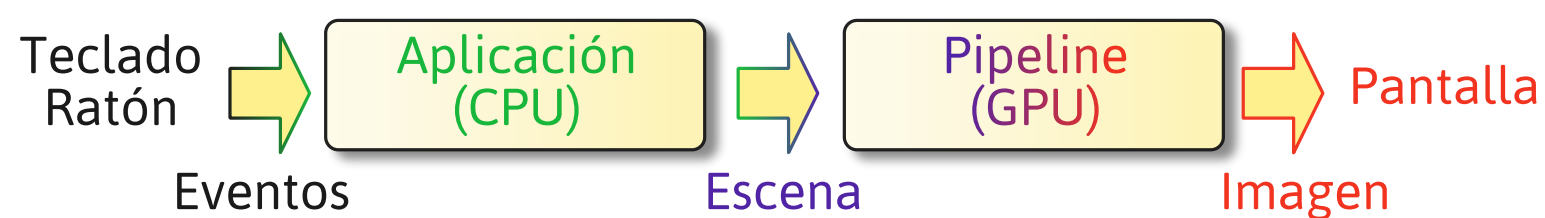
3. El tercer paso debe hacer dos cosas por cada fragmento: Determinar si efectivamente se ve (por ej: el fragmento podría ser de un triángulo que en la escena está detrás de otro que lo tapa); y calcular la iluminación (el color final de acuerdo al material del triángulito y qué pasa con la luz). Yodo esto ocurre en el mundo raster.

En realidad son más pasos, cuando estudiemos estas etapas en detalle las explotaremos en pasos más específicos.

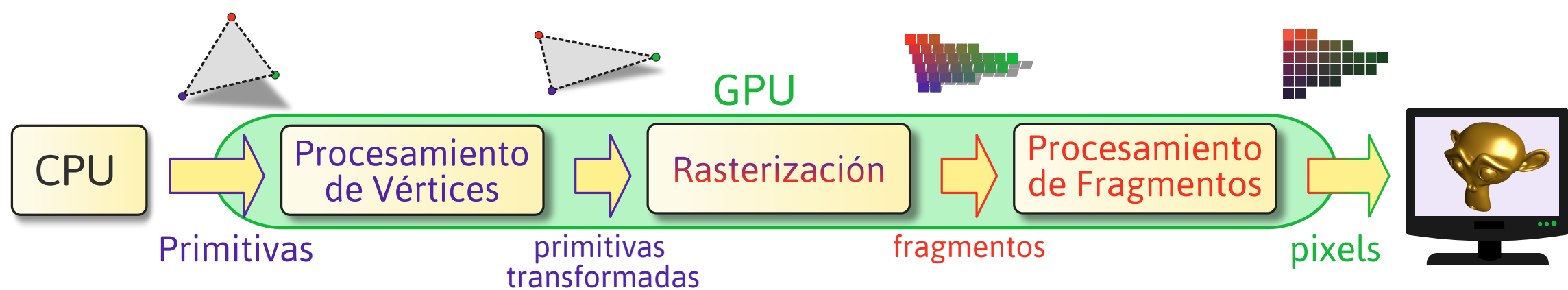
Las placas gráficas (y las APIs gráficas) han evolucionado durante casi 30 años por el camino de la rasterización. Están diseñadas para implementar estas etapas de forma eficiente

SISTEMA GRÁFICO Y PIPELINE

- Entrada / procesamiento / salida



- El **Pipeline** de renderizado (versión extendida):



Nuestro sistema gráfico de referencia, y para las prácticas, será una PC convencional.

Por entrada tendremos mouse y teclado. Otras entradas pueden ser un joystick, una pantalla táctil, un lápiz óptico o una tableta digitalizadora, los sensores de movimiento de un teléfono o de un casco de realidad virtual, etc.

Para el procesamiento una CPU estándar. Simplificando mucho contiene: memoria RAM como espacio de trabajo, uno o pocos núcleos muy generales para el procesamiento, almacenamiento persistente mediante un disco duro, SSD, o equivalente, hardware gráfico específico (la GPU), y un bus conectando todo.

Para la salida usaremos monitores. Otras salidas en otros sistemas gráficos pueden ser un plotter o una impresora, un archivo de imagen o video, otros tipos de monitores (como los de un caso de VR), etc.

Es importante notar que en general cuando decimos "el pipeline", pensamos en estas etapas que hacen directamente al renderizado, pero antes la CPU debió preparar la escena.

Agregaremos en nuestros esquemas una etapa previa llamada "Aplicación" que es la parte que corre en la CPU, detecta la interacción del usuario, y prepara los datos y la configuración para alimentar a la GPU.

Tomaremos este esquema a modo de referencia/mapa general para darle contexto a cada unidad y decir en qué etapa se aplica cada cosa que vamos aprendiendo.

LA GPU: UN POCO DE HISTORIA...

- 70s/80s/90s: Adaptadores de video (no ayudaban a renderizar)
 - Mediados de los 90s: Empiezan a "ayudar"
- 1999/2000: Nace la **GPU**
 - Pipeline de rasterización fijo
- 200X: Pipeline programable: **Shaders**
 - Etapas programables, cada vez más flexible
 - **GP-GPU**: minería de criptomonedas, inteligencia artificial, compresión descompresión de audio y video, física de videjuegos, etc.
- 2000-2020: Más memoria, más clock, más hilos, más instrucciones, mejor consumo, etc...
- 2018-????: Raytracing y ¿más I.A.?

En el principio (80s, 90s) estaban los adaptadores gráficos, placas cuya única funcionalidad era convertir la imagen en una señal analógica para los monitores de la época... Pero no colaboraban en la generación de la imagen

Durante los 90s empezaron a colaborar en la creación de la imagen 3D, implementando de a poco pasos del pipeline mediante hardware específico.

A comienzos de los 2000 aparecieron placas que contenían todo el pipeline fijo. Allí se las empezó a llamar GPU (graphics processing unit). Al principio ese pipeline era bastante rígido: como una gran máquina con etapas bien definidas, y solo algunas "perillas" para ajustar configuraciones menores en cada etapa. Ese pipeline fijo es el que vamos a tomar de referencia y estudiar en profundidad.

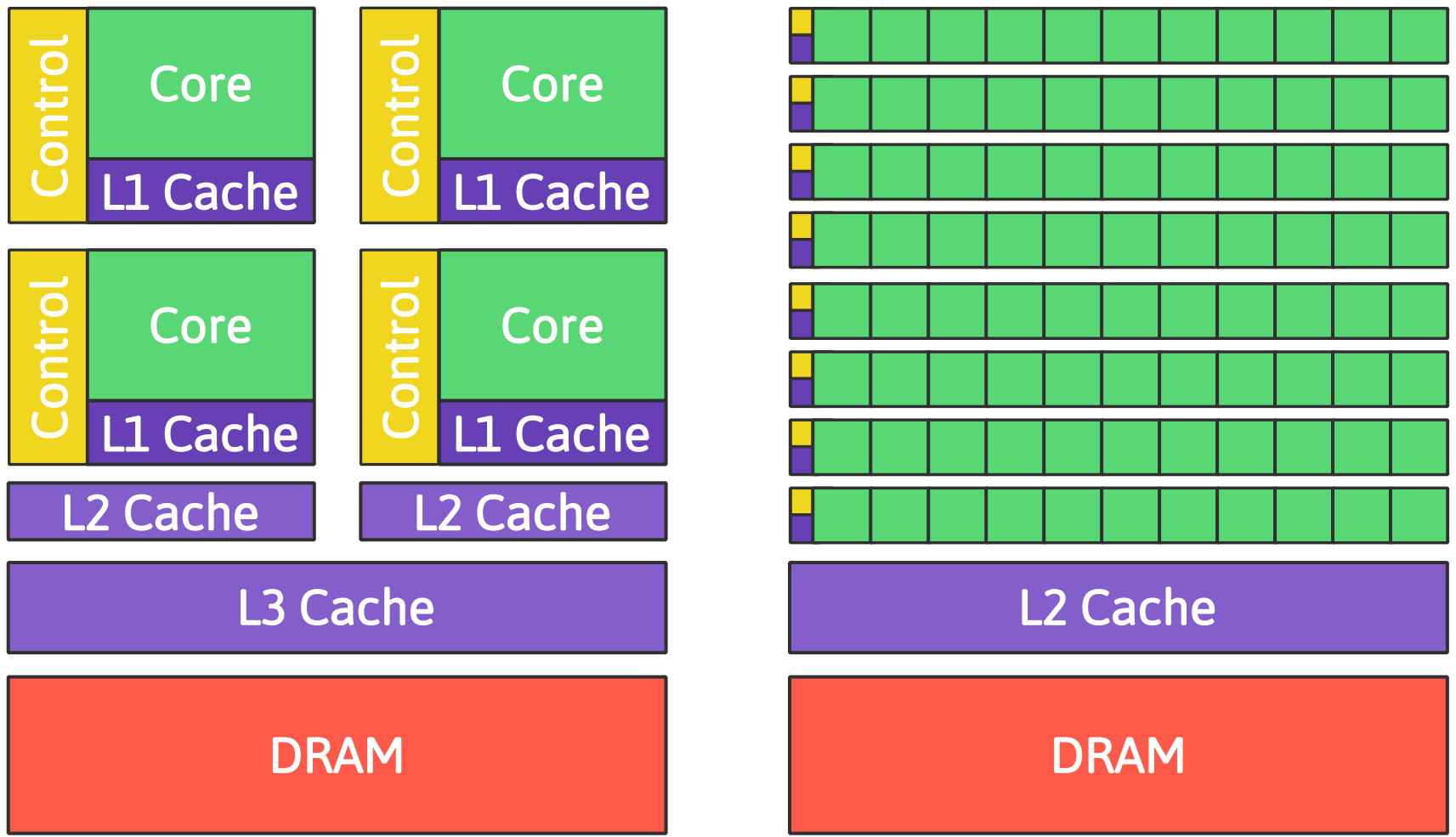
Entre 2000 y 2010 fueron ganando flexibilidad, hasta permitir reprogramar casi completamente las etapas principales. Estos programas que corren en la GPU son los llamados "shaders".

Al principio dentro de la GPU había hardware específico para cada etapa. Ahora hay núcleos más genéricos que pueden asignarse bajo demanda a distintas etapas. Muy pocos pasos siguen teniendo hardware dedicado (la etapa de rasterización y poquito mucho más).

Al ser estas unidades más flexibles, se empezaron a usar para más cosas. Aquí se empieza a hablar de GP-GPU (General Purpose GPU). No solo gráficos, también física, inteligencia artificial, o minado de criptomonedas. Se programan de una forma muy particular, no sirven para cualquier cosas, pero bien usado es mucha capacidad de cálculo.

A partir de 2018/2019 empiezan a incorporar etapas pensadas para un pipeline alternativo para raytracing. Pero estas placas todavía no son tan comunes, por el momento son muy caras, pero es la tendencia actual.

ARQUITECTURA: CPU VS. GPU



Una CPU actual tiene cierto grado de paralelismo: Pocos (2,4,6,8) núcleos muy complejos (CISC, MIMD); Nucleos relativamente independientes (cada uno con su propia lógica de control puede estar haciendo algo diferente al resto, MIMD).

Una GPU actual tiene un altísimo grado de paralelismo: Muchos cores (cientos, o miles) muy simples (RISC); Por grupos, los núcleos comparten la lógica de control, por lo que están obligados a hacer lo mismo al mismo tiempo (SIMD).

Programar para las GPUs es más difícil (bajo nivel) y requiere de lenguajes específicos (GLSL, Cuda, HLSL, la mayoría "inspirados" en C/C++)

¿QUÉ HAY QUE SABER?

- Conceptos básicos: **escena, primitiva, rendering**
- Tipos de rendering: **realista vs. NPR, offline vs. real-time.**
- Formas de representación: **vector vs. raster.**
- Lineamientos general de las dos familias de algoritmos.
 - **rasterización vs. raytracing**
- Etapas principales del **pipeline de rasterización.**